

Formal Verification of Lightning Network Multi-Hop HTLC Routing: From Concrete Models to Arbitrary Path Lengths

First Author
University XYZ
City, Country
email@university.xyz

Second Author
University XYZ
City, Country
email@university.xyz

Third Author
Institute of Technology, ABC
City, Country
email@abc.edu

ABSTRACT

The Lightning Network (LN) relies on Hashed Time-Locked Contracts (HTLCs) and complex channel state machines to enable scalable, multi-hop blockchain payments. Prior symbolic analyses have targeted either two-party HTLC settings or generic message-forwarding abstractions; the interaction between the channel revocation lifecycle, multi-hop routing, and block-height timing has not been mechanized as a single symbolic model. We present such a model in the Tamarin prover, structured as a modular package of five theories establishing **56 distinct machine-checked properties** (81 lemma checks in total, since 25 properties are additionally re-proved in the N -hop abstraction), covering the two-party channel lifecycle, multi-hop HTLC routing with fees, staggered-CLTV timing, and an N -hop abstraction.

Our central finding is a consequence of that integration. We identify a *linear single-spend discipline*: two independent soundness gaps—HTLC output duplication and revoked-commitment replay—share one root cause, namely modeling one-shot on-chain resources as Tamarin *persistent* facts rather than *linear* ones, and both are repaired by the same linear-token remedy. The second instance is visible only when revocation and routing are modeled together, which narrower prior models structurally could not observe; we propose the resulting invariant as a reusable modeling discipline for symbolic payment-channel analyses. We further prove an end-to-end value-conservation theorem and expose a griefing DoS; establish *assumption minimality* empirically, showing that removing the intermediary-claim restriction yields a concrete 59-step counterexample; and reproduce the known wormhole (agent-skipping) attack as a fidelity check on the model. Finally, we identify Tamarin’s origin-analysis recursion on self-referential signed messages as a barrier to unbounded verification, and give a proof-layering technique—machine-checked local authentication plus a pen-and-paper refinement theorem—that transfers core routing invariants to paths of arbitrary length.

CCS CONCEPTS

• **Security and privacy** → **Formal methods and theory of security**; *Cryptography*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference’17, July 2017, Washington, DC, USA

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
<https://doi.org/10.1145/0000000.0000000>

KEYWORDS

Formal verification, Tamarin prover, Lightning Network, Payment Channels, HTLC, Blockchain, Layer-2 scaling

ACM Reference Format:

First Author, Second Author, and Third Author. 2027. Formal Verification of Lightning Network Multi-Hop HTLC Routing: From Concrete Models to Arbitrary Path Lengths. In . ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/0000000.0000000>

1 INTRODUCTION

Blockchain technologies have revolutionized decentralized finance, but their inherent reliance on global consensus creates a fundamental scalability bottleneck. To achieve the transaction throughput required for global adoption, Layer-2 (off-chain) scaling solutions have become essential. The Lightning Network (LN) [16] is the most prominent of these, enabling fast, low-fee payments by moving the bulk of transactions off-chain and settling only disputes on the underlying blockchain [8].

At the heart of the Lightning Network are Payment Channels and Hashed Time-Locked Contracts (HTLCs) [1]. HTLCs enable multi-hop payments across a network of untrusted intermediaries by conditioning the release of funds on the knowledge of a secret preimage, combined with staggered absolute timelocks (CLTV). Alongside this, the channel lifecycle relies on a complex revocation and punishment mechanism to prevent participants from publishing outdated, fraudulent channel states [10].

While the conceptual design of LN multi-hop routing is elegant, the interaction between cryptographic primitives, block-height timing constraints, and off-chain state machines is highly non-trivial. Subtle edge cases—such as early-timeout races where an incoming channel refunds before an outgoing channel is redeemed—can compromise the safety of honest intermediaries. Furthermore, security in these systems is heavily dependent on liveness assumptions: honest nodes must monitor the network and act before their HTLCs expire.

Existing formal treatments have addressed parts of this stack in isolation. Symbolic analyses in Tamarin have targeted two-party HTLC constructions such as cross-chain swaps [2] and contingent payments [3]; generic symbolic frameworks have studied path integrity and agent-skipping in message-forwarding protocols [17]; and model checking with TLA+ has verified balance security for Lightning channels and bounded-length multi-hop payments [7]. What has not been mechanized is the *combination*: the two-party revocation and punishment lifecycle together with multi-hop HTLC routing, per-hop fees, and staggered CLTV timing, in one symbolic model under a full Dolev–Yao adversary. To our knowledge, the

model we present is the first Tamarin formalization to integrate these layers.

That integration is not merely a matter of coverage; it is what makes our central finding observable. We identify a category error—modeling one-shot on-chain resources as *persistent* rather than *linear* Tamarin facts—that manifests twice, once in the HTLC layer and once in the revocation layer. The second instance involves a rule gated by revocation state being fired repeatedly along a routing execution, and is therefore invisible to models that contain only one of the two layers.

To conduct this analysis we use the Tamarin prover [14] under a full Dolev–Yao adversary [6]. We explicitly model integrity and safety; we do not model payment privacy or unlinkability (a confidentiality property requiring onion routing and observational-equivalence proofs), stated here as an explicit exclusion. Comprehensively verifying the Lightning Network introduces severe methodological and tooling hurdles. First, the protocol inherently requires the combination of Dolev–Yao cryptographic reasoning, natural-number induction (for CLTV arithmetic), and unbounded block clocks. As we demonstrate, attempting to verify these aspects in a single monolithic theory causes state-of-the-art provers to exhaust available memory (OOM). Second, while prior analyses often fix the routing path to a specific length, a natural question arises: do these safety properties hold for paths of arbitrary length (N hops)?

In this paper, we present a comprehensive, modular formal verification of Lightning Network multi-hop HTLC routing. Our contributions are as follows:

- (1) A **linear single-spend discipline**: a category error in which one-shot on-chain outputs are modeled as persistent facts, identified in two independent instances (HTLC output duplication and revoked-commitment replay) and repaired uniformly by structural linear-resource guards, replacing global axioms. We propose the resulting invariant—*every one-shot on-chain output is a linear fact*—as a reusable modeling discipline for symbolic payment-channel analyses.
- (2) A modular, fully machine-checked model integrating the two-party channel lifecycle (including revocation and punishment) with multi-hop HTLC routing and fees, establishing **56 distinct properties** across five theories (81 lemma checks, as 25 properties are re-proved in the N -hop abstraction). This integration is the precondition for the finding above. We report the composition of this count explicitly in Table 1 rather than quoting a single aggregate.
- (3) An empirical assumption-minimality result proving that removing the intermediary-claim restriction yields a concrete 59-step counterexample, together with a two-sided certificate that the corresponding liveness assumption is load-bearing.
- (4) Economic-soundness theorems (end-to-end value conservation, strictly positive per-hop fees) and a reachable griefing DoS, plus a reproduction of the known wormhole (agent-skipping) attack [12, 17] as a fidelity check on the model.
- (5) A map of the prover’s tractability boundaries, identifying the precise bounding factors (the signing/nat/clock OOM

triple, signed-message N -hop routing, and reusable channels), and a proof-layering technique—machine-checked local authentication plus a pen-and-paper refinement theorem—for N -hop invariants.

2 RELATED WORK

2.1 Formal Verification of Payment Channels

Early formalizations with idealized channel closures [5, 11, 15] successfully proved payment atomicity and balance conservation but omitted the revocation and punishment mechanisms required to secure real-world implementations.

Prior symbolic analyses have applied Tamarin to HTLC-based cross-chain swaps [2] and zero-knowledge contingent payments over a public ledger [3]. These models target two-party settings and lack multi-hop routing, channel lifecycle revocation, or staggered CLTV arithmetic. Our work explicitly bridges this gap within a unified, automated framework targeting the multi-hop setting.

2.2 Agent-Skipping and Path Integrity

Smith et al. [17] develop a general symbolic framework, in the multiset-rewriting setting, for *agent-skipping* attacks on message-forwarding protocols, introducing path-integrity goals and analyzing several protocols including payment channel networks. The wormhole attack on HTLC routing [12] is an instance of agent-skipping, and their framework establishes it as a symbolic reachability result. Our treatment of the wormhole is therefore *not* a novel attack finding but a fidelity check: we confirm that our integrated model reproduces the known attack, and additionally tie it to the value layer by quantifying the theft to exactly the bypassed routing fees within a model that also carries channel revocation and settlement. The complementary distinction is one of scope: Smith et al. abstract the forwarding protocol to study path integrity generically, whereas we model the concrete Lightning stack—revocation, punishment, fees, CLTV—in order to reason about properties that depend on those layers.

2.3 Model Checking and Computational Treatments

Grundmann and Hartenstein [7] formalize Lightning in TLA+ and model-check balance security, using stepwise refinement and a time abstraction to make single channels and multi-hop payments checkable separately, scaling to payments over up to four hops with two concurrent payments. Their approach is complementary to ours along two axes. Methodologically, model checking explores a bounded state space with an explicit adversary encoding, whereas symbolic verification in Tamarin reasons over unbounded sessions under an equational Dolev–Yao model; consequently their analysis handles concurrency that we do not, while ours handles unbounded message-derivation capabilities and, via the abstraction of Section 7, unbounded path length. Substantively, their refinement establishes that the current Lightning specification satisfies balance security; our contribution is orthogonal, concerning a modeling pitfall in the symbolic encoding of one-shot on-chain outputs.

Kiayias and Litos [9] provide a composable, game-based (UC) security treatment of the LN at the two-party channel layer; it

does not machine-check multi-hop HTLC routing. Our symbolic approach achieves a high degree of automation—establishing 56 distinct machine-checked properties—while explicitly covering multi-hop routing under a full Dolev–Yao adversary. Prior work frequently treats the liveness requirement of intermediaries as an implicit assumption. The Tamarin analysis of HTLC swaps by Boyd et al. [2] surfaced a timing-related hidden assumption (stable relative chain growth), which parallels our finding that liveness assumptions are strictly load-bearing; we provide a mechanized, two-sided certificate for this assumption in a model that includes the full channel lifecycle.

2.4 Tamarin in Blockchain Protocols

The Tamarin prover [14] is a standard tool for verifying cryptographic protocols, successfully applied to industrial standards such as TLS 1.3 [4]. In the blockchain domain, Tamarin has been applied to HTLC swaps and contingent payments [2, 3]. Existing Tamarin applications typically verify fixed-round protocols. Applying Tamarin to stateful, long-running protocols with unbounded loops introduces unique challenges. We do not merely restrict the model to avoid OOM; we rigorously diagnose the root causes, pinpointing Tamarin’s origin-analysis recursion on self-referential signed terms as the fundamental barrier to verifying generic N -hop routing.

3 PROTOCOL OVERVIEW AND THREAT MODEL

3.1 Payment Channel Lifecycle

A payment channel is established via an on-chain funding transaction. Once opened, parties transact off-chain by exchanging signed commitment transactions. To prevent a malicious party from broadcasting an outdated state, LN employs a revocation mechanism. When transitioning to a new state, parties exchange revocation secrets. If a revoked state is published, the other party can unilaterally punish the cheater and claim the entirety of the channel funds, protected by a relative timelock (CSV). Channels can be closed cooperatively or unilaterally.

3.2 Multi-Hop HTLC Routing

To pay a recipient without a direct channel, LN uses HTLCs to route payments across intermediate nodes. Assume a path $Sender \rightarrow F_1 \rightarrow \dots \rightarrow F_n \rightarrow Receiver$.

- (1) **The Lock Round (Setup):** The Receiver generates a random secret preimage x and sends its hash $y = H(x)$ to the Sender. The Sender adds an HTLC to their channel with F_1 , locking funds conditional on providing x before an absolute deadline. This cascades down, with each forwarder taking a routing fee.
- (2) **The Release Round (Settlement):** The Receiver reveals x to claim the final HTLC. F_n uses x to claim the HTLC from F_{n-1} . This cascades backward to the Sender.

3.3 Threat Model and Security Goals

We adopt the standard Dolev–Yao adversary model [6], representing a network attacker with full control over the communication

medium but who cannot break cryptographic primitives. We explicitly model long-term key compromise as a distinct adversarial capability.

The Liveness Assumptions. Security degrades into a liveness requirement. Our model encodes four timing restrictions:

- **T1** (RedeemBeforeTimeout);
- **T2a** (ForwardTimeGap);
- **T2b** (HonestPartiesActBeforeIncomingTimeout);
- **T3** (IntermediaryMustClaim).

These are not on an equal footing. T1 and T2a/T2b are *passive temporal orderings*: they constrain when events may occur relative to one another, and correspond to the CLTV structure that Section 6 derives independently from a concrete block clock. T3 is different in kind—it obliges an intermediary to *act*, sweeping its incoming HTLC once the outgoing one is redeemed. It is therefore the assumption most at risk of being an unnoticed strengthening, and the one we subject to minimality testing in Section 6. We rely on all four for our main theorems and do not claim minimality for the passive orderings. This mirrors the real-world necessity of LN watchtowers [13]. Our core security goals translate directly to verified lemmas: Payment Atomicity, Intermediary Safety, Routing Causality & Authentication, Timing Safety, and Value Conservation.

4 FORMAL PRELIMINARIES

We fix notation for Tamarin’s multiset-rewriting semantics [14], which the remainder of the paper uses to state both the modeling discipline of Section 6.1 and the refinement theorem of Section 7.

Terms and facts. Let $\mathcal{T}$ be the set of terms over a signature with an equational theory E (here: hashing, signing, and natural numbers). A *fact* is $F(t_1, \dots, t_k)$ with $t_i \in \mathcal{T}$. The set of fact symbols is partitioned as $\Sigma_{\text{fact}} = \Sigma_{\text{lin}} \uplus \Sigma_{\text{per}}$ into *linear* and *persistent* symbols; we write persistent facts with a leading bang, e.g. $!\text{RevokedSecret}(\cdot)$. A *state* S is a finite multiset of ground facts.

Rules and transitions. A rule is $ri = l \xrightarrow{a} r$ with l, a, r multisets of facts (a the *action* facts). Writing $\text{lin}(l)$ and $\text{per}(l)$ for the linear and persistent premises, the transition relation is

$$S \xrightarrow{a} (S \setminus^{\#} \text{lin}(l)) \cup^{\#} r \quad \text{provided} \quad \text{lin}(l) \subseteq^{\#} S \wedge \text{per}(l) \subseteq S,$$

where $\setminus^{\#}, \cup^{\#}, \subseteq^{\#}$ are multiset operations. The asymmetry is the point: linear premises are *removed* from the state, while persistent premises are merely *checked* and remain available. A *trace* is the sequence $tr = [a_1, \dots, a_n]$ of action multisets of an execution from the empty state; $\text{traces}(R)$ denotes the traces of a rule set R . Properties are first-order formulas over $F(\bar{i})@i$ with timepoint ordering $i < j$, interpreted over traces in the standard way; $R \models_{\forall} \varphi$ means all traces satisfy φ and $R \models_{\exists} \varphi$ that some trace does.

Restrictions. A restriction ρ filters the trace set: verification is carried out over $\text{traces}_{\rho}(R) = \{tr \in \text{traces}(R) \mid tr \models \rho\}$. Restrictions are therefore *assumptions*, not theorems, which is why Section 6 tests whether one of them is load-bearing.

5 FORMAL MODELING METHODOLOGY

5.1 The Curse of Monolithism and Modular Decomposition

A naive approach is to construct a single, monolithic theory containing the channel lifecycle, HTLC routing logic, CLTV block-height arithmetic, and value conservation. This is fundamentally intractable. The combination of the signature equational theory, natural-number induction, and an unbounded block clock causes Tamarin's search space to explode. Crucially, this was tested, not assumed—any two of the three ingredients remain tractable; only the full triple OOM-kills the prover.

The model is therefore split so that no single theory carries all three. The architecture is summarized in Table 1.

5.2 Tactic Engineering and Reproduction

All results were produced under a strict test-first discipline: a property is claimed only after Tamarin reports it. Per-lemma invocation is used on the large theory to avoid OOM. The flag `--stop-on-trace=seqdfs` rescues the heavy existence witnesses but hangs the natural-number arithmetic theories, so it is applied selectively.

6 VERIFICATION OF THE CORE MODEL

Across the theories of Table 1 we establish 56 distinct properties (81 lemma checks in total). We stress that the N -hop theory contributes no new properties: every one of its 25 lemmas restates a property already proved in `multihop.spthy`, re-established over the abstract routing layer. Of the 43 lemmas of the concrete theory, 10 are exists-trace witnesses (including non-vacuity checks and the attack traces of Sections 6.1 and 6.2) and 5 are `[reuse]` helper lemmas that support other proofs rather than stating protocol goals; the complete lemma index is given in the artifact. Table 2 maps the lemma groups in our primary concrete theory.

6.1 The Linear Single-Spend Discipline (Axiom to Structure)

We now state formally the pitfall informally described above, using the notation of Section 4.

Definition 6.1 (One-shot resource). A fact symbol Res denotes a one-shot resource if, in the intended protocol semantics, for every ground instantiation $\bar{t}$ the privilege represented by $\text{Res}(\bar{t})$ may be exercised at most once. On-chain outputs—a funding output, an HTLC output, the output of a revoked commitment—are one-shot resources.

The following observation is immediate from the transition relation—a persistent premise is checked but not removed—and we state it only to fix the shape of the defect precisely. The contribution is *not* this fact about Tamarin, which is standard, but the identification of which Lightning constructs fall under Definition 6.1 and what the concrete exploit is when they do not.

PROPOSITION 6.2 (PERSISTENT GATING ADMITS UNBOUNDED EXERCISE). Let $ri = (P \uplus \{\text{!Res}(\bar{t})\}) \xrightarrow{a} r$ be a rule in which the only premise representing the resource is the persistent fact $\text{!Res}(\bar{t})$, and suppose the remaining premises P are persistent or otherwise

re-derivable. Then for every $n \in \mathbb{N}$ there is a trace of R containing at least n occurrences of the action a under the same instantiation $\bar{t}$.

PROOF. Immediate: firing ri removes only $\text{lin}(I)$, and $\text{!Res}(\bar{t}) \in \Sigma_{\text{per}}$ is checked but not removed, so with P re-derivable the enabling condition holds again in the successor state. $\square$

Remark 1 (Where the difficulty actually lies). That linear facts model consumable resources is elementary, and experienced Tamarin users apply it routinely to nonces, session keys, and state-machine transitions. The difficulty is not the mechanism but the *mapping*: deciding which constructs of a payment-channel protocol denote one-shot on-chain outputs. This is not syntactically evident. A revoked commitment presents itself in the protocol description as *knowledge*—“the counterparty learns the revocation secret”—and knowledge is exactly what persistent facts are for; the natural encoding is therefore the wrong one. What is easy to miss is that the same revocation event confers two distinct things: durable knowledge of the secret, and a single opportunity to spend the revoked output. Our contribution is the identification of this split, and the demonstration below that getting it wrong yields not a modeling infelicity but an exploitable trace.

Proposition 6.2 is exactly the defect we found twice. In Instance 1 the resource is the channel output hosting an HTLC and a is an HTLC-add; in Instance 2 the resource is a revoked commitment, $\text{!Res} = \text{!RevokedSecret}$, and $a = \text{PublishRevoked}$, so the protocol admits arbitrarily many punishments for a single cheat.

Definition 6.3 (Linear single-spend token). A single-spend token for a one-shot resource is a linear fact symbol $\text{Tok} \in \Sigma_{\text{lin}}$ together with a discipline requiring (i) every rule exercising the resource to take $\text{Tok}(\bar{t})$ as a linear premise, and (ii) $\text{Tok}(\bar{t})$ to be produced at most once per $\bar{t}$ across all rules.

PROPOSITION 6.4 (STRUCTURAL SINGLE SPEND). Under Definition 6.3, for every trace and every $\bar{t}$ the action a occurs at most once with instantiation $\bar{t}$.

PROOF. Let p be the number of productions and c the number of consumptions of $\text{Tok}(\bar{t})$ in a trace. As Tok is linear, each firing of a rule exercising the resource removes one copy from the state, so $c \leq p$; by (ii) $p \leq 1$, hence $c \leq 1$. By (i) each occurrence of a under $\bar{t}$ is such a firing, so a occurs at most once. $\square$

Concretely, $\text{Free}(ptr)$ is produced once by `Lock_Funds_And_Open` (one per channel open) and consumed by every HTLC-add, and `RevokedCommitmentUnspent(ptr, A, B, n, commit)` is produced once by `Revoke_Old_Secret` and consumed by `Publish_Revoked`:

```
Publish_Revoked_A :
[ !ChannelRecord(ptr, A, B), !ValidOnChain(ptr),
  !RevokedSecret(A, n, s, c),
  RevokedCommitmentUnspent(ptr, A, B, n, c) ]
  PublishRevoked(A, n, c), Cheat(A, n)
  [ OnChainCheat(A, B, n, c), RevealedSecret(A, n, s, c) ]
```

Persistent *knowledge* of the revocation secret is retained via `!RevokedSecret`; only the one-shot *spend* is linear. This separation is the content of the discipline.

Table 1: The five-theory modular architecture. “Safety” counts all-traces properties and “Witness” counts exists-trace lemmas (reachability results, attack witnesses, and non-vacuity checks). The N -hop theory re-proves properties already counted in multihop.spthy rather than adding new ones, so the distinct total is 56 and the total number of lemma checks is 81.

Theory File	Aspect Modeled	Safety	Witness	Builtins
multihop.spthy	Lifecycle, 3-hop routing, value/fees, wormhole	33	10	Hashing, Signing, Nat
Clock.spthy	Block-clock lifecycle & staggered-CLTV claim windows	6	3	Natural-numbers
cltv_blocks.spthy	Pure CLTV block arithmetic (delta positivity)	3	0	Natural-numbers
timeout.spthy	Early-timeout race (T2b removed)	0	1	—
<i>Distinct properties</i>		42	14	
multihop_nhop.spthy	<i>Re-proof</i> of the above in the N -hop abstraction	19	6	Hashing, Signing, Nat

Table 2: Lemma groups in multihop.spthy (43 total).

Group	Class	Representative Lemmas
Invoice / preimage auth	Safety	Invoice_Authenticates_Settlement
Lock causality	Safety	Forward1_Requires_Offer
Release causality	Safety	Fulfill_Requires_Forward2
Atomicity / non-vacuity	Safety/Reach.	Payment_Atomicity_Under_Liveness
Value & fees	Safety	EndToEnd_Value_Conservation
Refund / grieving	Reachable	Honest_Intermediary_Refunds
Wormhole	Attack	Wormhole_Steals_Exactly_The_Fees
Intermediary safety	Safety	Intermediary_Never_Loses_Under_Liveness
Channel lifecycle	Safety	No_Punishment_Without_Cheating

Remark 2 (Why re-verification is required). Adding a premise shrinks the enabling condition, so $\text{traces}(R') \subseteq \text{traces}(R)$. Hence every $\forall$ -property is preserved: $R \models_{\forall} \varphi \Rightarrow R' \models_{\forall} \varphi$. The converse fails for $\exists$ -properties, since a witness may be removed. We therefore re-ran the full theory rather than appealing to monotonicity; all 43 lemmas re-verify (262 s, Tamarin 1.8).

Instance 1: HTLC output duplication. The forwarding rules were gated only by a public HTLC-offer message. A Dolev-Yao replay of that offer could re-fire an honest forward, locking a second HTLC on the same channel output with an adversary-chosen amount. We replaced a global `OneHTLCPerPtr` axiom with the structural guard of Definition 6.3: each channel open emits one linear `Free(ptr)` token, consumed by every HTLC-add.

Instance 2: revoked commitment replay. The punishment rules were gated only by the persistent `!RevokedSecret` fact, and are thus an instance of Proposition 6.2. The consequence is concrete rather than cosmetic. A revoked commitment corresponds to one on-chain transaction spending one funding output; the counterparty’s penalty claim spends the resulting output once. Under persistent gating, however, a single instance of cheating licenses `PublishRevoked`—and hence `Punish`—arbitrarily many times for the same revoked state n , so the model admits an execution in which the honest party confiscates the channel balance repeatedly from one cheat. A model in which punishment is unbounded does not merely overstate the honest party’s position: any downstream analysis of balance security, of grieving economics, or of watchtower incentives inherits an adversary-favourable or victim-favourable distortion depending on which party is compromised.

The defect is silent because every individual lemma about punishment soundness (`No_Punishment_Without_Cheating`) continues to hold—punishment still requires a prior cheat; it is only the *multiplicity* that is wrong.

We stress that Instance 2 is a consequence of integration rather than of scale. It resides at the junction of the revocation lifecycle and the routing execution: the offending rule is gated by revocation state yet re-fired along a routing trace. A model containing only the two-party channel layer, or only an abstracted forwarding layer, does not exhibit the trace that exposes it.

The reusable invariant. Definitions 6.1–6.3 and Propositions 6.2–6.4 yield a mechanical audit question for symbolic payment-channel models: *for each fact denoting a one-shot on-chain output, is there a linear token consumed when it is exercised?* Persistent facts may encode durable knowledge (keys, revealed secrets, channel identity), but never a spend opportunity.

6.2 Wormhole Attack: A Fidelity Check

The wormhole attack [12] occurs when two colluding path members relay the preimage out-of-band, skipping an honest intermediary who forwards value but earns no fee. It is an instance of the agent-skipping class formalized symbolically by Smith et al. [17], and we claim no novelty for exhibiting it. Its role here is to discharge a specific obligation created by Section 6.1. Having constrained the model with two linear single-spend tokens, we must show that the repair is *sound* rather than merely restrictive: a guard that eliminates the double-punishment trace by over-constraining the protocol would also eliminate legitimate executions, and the resulting model would verify safety vacuously. Reproducing the wormhole demonstrates the opposite—the hardened model still

admits the known economic pathologies of HTLC routing, so the linear tokens remove exactly the spurious multiplicity and nothing else. It is, in this sense, a non-vacuity argument for the central contribution rather than an attack result in its own right. In the symbolic model, the Dolev–Yao network is the colluders’ side channel. We machine-check it twice: `Wormhole_Fee_Theft_Reachable` (51 steps) exhibits a trace where the payment settles yet the middle hop is never redeemed and no intermediary earns a fee, with no key compromise; and `Wormhole_Steals_Exactly_The_Fees` (81 steps) quantifies the theft to precisely the path fees $\%f1 + \%f2$, tying the attack to the value layer of a model that simultaneously carries channel revocation and settlement. Known mitigations (AMHL, PTLCS) are out of scope.

6.3 Economic Soundness and Griefing

`EndToEnd_Value_Conservation` establishes that the sender locks the receiver’s amount plus the accumulated fees, so the routing mechanism neither creates nor destroys value. As a reachability result, `Honest_Intermediary_Refunds` exhibits an honest intermediary induced to lock capital and then refund on timeout—the standard PCN grieving DoS.

Remark 3 (Zero-fee forwarding is out of scope). The lemmas `Fee_Strictly_Positive_Hop` hold in our model for a reason that is an artifact of the encoding rather than a property of Lightning: Tamarin’s natural-numbers sort provides no $\%0$ term, so a zero-fee forward is not expressible. We therefore do *not* present these as economic theorems. Deployed Lightning explicitly permits zero base fees, which nodes use for re-balancing and probing; our model consequently verifies a *restricted subset* of the protocol in which every hop charges a positive fee, and cannot evaluate routing strategies that depend on zero-fee forwards. Admitting zero fees requires either a different arithmetic encoding or an explicit zero constant with the associated case analysis, which we leave to future work. We note that the value-conservation and atomicity results do not depend on fee positivity; only the fee-specific lemmas do.

6.4 Timing Safety and Assumption Minimality

`cltv_blocks` derives the CLTV-delta invariant from concrete block numbers; `Clock` adds block-clock lifecycle safety; and `timeout` exhibits the early-timeout race that reappears once T2b is removed.

Among our four restrictions, T3 is the one that encodes an obligation to act rather than a temporal ordering, and is accordingly the one whose necessity is least self-evident: it is natural to suspect it is implied by the orderings T1 and T2b. We tested it by removing T3 and re-running the suite: `Intermediary_Never_Loses_Under_Liveness` is falsified, with Tamarin finding a concrete 59-step counterexample. T2b orders the two timeouts but does not force the node to sweep the incoming HTLC once its outgoing one is redeemed; that action is exactly what T3 encodes. T3 is therefore **load-bearing** for intermediary safety: it is not redundant given the temporal orderings, because no ordering constraint can compel an action. We tested the removal of T3 specifically because it is the only one of the four that is an obligation rather than an ordering; we make no claim regarding the individual minimality of T1, T2a, or T2b, whose content is independently corroborated by the block-clock derivation of Section 6.

7 AN ABSTRACTION FOR UNBOUNDED PATH LENGTH (N -HOPS)

Replacing the hardcoded forward rules with one generic rule was attempted in several encodings. A generic signed-message forward causes Tamarin to OOM even for a 1-hop witness due to self-referential origin-analysis recursion.

To bypass this, we abstract the network layer into an internal Tamarin linear fact—`Route(prev, me, ptr, y)`—handed directly from one hop to the next. Because the HTLC is no longer a self-referential signed term, Tamarin’s origin analysis no longer recurses.

We are deliberate about what this buys and what it costs. Internalising the network *removes the Dolev–Yao adversary from the routing layer*: in the abstract model the attacker can no longer inject, replay, or forge a forwarded HTLC. What the abstract model therefore establishes at unbounded N is that the *internal state machine* of routing—the discipline of linear HTLC slots, the pairing of redeem and refund outcomes, and the timing restrictions—remains consistent along a chain of any length. It does *not* establish network-level adversarial security at N hops; that content resides entirely in the concrete model and reaches arbitrary N only through the pen-and-paper argument of Theorem 7.4, whose status we delimit in Remark 4. Readers should regard this section as a tractability workaround with a precisely stated transfer condition, not as a machine-checked security result for unbounded path length. This yields `multihop_nhop.spthy` (25 active lemmas; 4 `[use_induction]` lemmas are commented out as deliberate tractability boundaries because backward chaining through unbounded `Route` facts causes divergence).

Fees are enforced structurally in the N -hop model as a firing guard (`Eq(%vIn, %vOut + %fee)`), meaning end-to-end conservation over N hops is a sum of N fees—not finitely statable in one Tamarin lemma. We note also that the abstraction internalises the network, and therefore deliberately does *not* exhibit the wormhole: the side channel the attack exploits is absent by construction, which is why that result is established in the concrete model.

7.1 Refinement Theorem

Internalising the network removes Dolev–Yao power, so the abstract model A has strictly fewer behaviours than the concrete model C ; abstract safety therefore does not by itself imply concrete safety. We make the transfer precise.

Definition 7.1 (Shared alphabet and projection). Let

$\Sigma = \{ \text{ForwardHTLC, Redeemed, Refunded, LtkCompromised} \}$ be the safety-relevant action symbols, emitted with identical arities by both theories. The projection $\pi : \text{traces}(C) \rightarrow (2^\Sigma)^*$ retains every Σ -action verbatim, deletes all network and cryptographic events (`Out`, `In`, `KU`, signature checks), and relabels the position-indexed `HTLCForwardedi` to the generic `HTLCForwarded`.

Definition 7.2 (AUTH). C satisfies `AUTH` if every forwarding step is preceded by a genuine offer or forward from the *adjacent* previous hop, unless that hop’s key is compromised; for the first position,

$$\begin{aligned} \forall F_1, F_2, S, p, y, v_{\text{in}}, v_{\text{out}}, i. \text{HTLCForwarded}_i(F_1, F_2, S, p, y, v_{\text{in}}, v_{\text{out}}) @ i \Rightarrow \\ (\exists p', j. \text{HTLCOffered}(S, F_1, p', y, v_{\text{in}}) @ j \wedge j \leq i) \\ \vee (\exists k. \text{LtkCompromised}(S) @ k \wedge k < i), \end{aligned}$$

and symmetrically for the second position.

AUTH is machine-checked in C as `Forward1_Requires_Offer` (266 steps) and `Forward2_Requires_Forward1` (274 steps). Crucially it quantifies over a *single adjacent pair* and never over the path or N ; this locality is what makes it provable, and its failure for a generic signed rule is the boundary reported in Section 7.

LEMMA 7.3 (PER-HOP SIMULATION). *Assume $C \models_{\forall} \text{AUTH}$. If $S \xrightarrow{a} S'$ is a forwarding step of C , then π of that step is a legal step of A .*

PROOF SKETCH. An honestly authenticated incoming HTLC is precisely what licenses the abstract `Route(prev, me, ptr, y)` hand-off, so the premises of the abstract `Forward_HTLCL` are met. The remaining case, a forged or replayed forward, is excluded by AUTH except under key compromise, which both models expose identically via `LtkCompromised` $\in \Sigma$ and which π preserves. $\square$

THEOREM 7.4 (SOUNDNESS OF THE ABSTRACTION). *Let φ be a safety property over Σ with $A \models_{\forall} \varphi$ for all path lengths N , and assume $C \models_{\forall} \text{AUTH}$. Then $C \models_{\forall} \varphi$.*

PROOF SKETCH. By induction on the number of forwarding steps of $tr \in \text{traces}(C)$. The base case is immediate. For the step, Lemma 7.3 extends $\pi(tr)$ by one legal abstract step, so $\pi(tr) \in \text{traces}(A)$. Since π preserves Σ -actions verbatim and φ is over Σ , any violation of φ in tr appears in $\pi(tr)$, contradicting $A \models_{\forall} \varphi$. $\square$

Remark 4 (Scope of mechanization). Both hypotheses of Theorem 7.4 are machine-checked—AUTH in C , and φ in A for unbounded N —but the composition is **pen-and-paper**: Tamarin can neither quantify over N nor relate two theories, for the same reason that key-management meta-theorems are argued externally. Soundness rests on the locality of AUTH (Definition 7.2), which we confirmed by inspecting every forward rule: each references only its adjacent pair. This is the honest boundary of our verification.

8 DISCUSSION: WHAT FAILED AND TRACTABILITY BOUNDARIES

We record negative results explicitly—they are a core methodological contribution.

- **The Three-Ingredient OOM.** Signing + natural-number induction + an unbounded block clock together exhaust the prover; any two are fine. This is the reason for the modular split.
- **Signed-Message N -Hop Routing.** Origin analysis on the self-referential ‘htlc’ term does not terminate, even for reachability, even with a fresh ID.
- **A Rejected Restriction.** An initial fix `OneRedeemPerPtr` (“at most one redeem per output”) was tested and rejected: it falsified the honest witness `MultiHop_Payment_Possible`, because the two Redeemed events in an honest run are the two endpoints’ views of one hop, not two spends. The correct guard was `OneHTLCPerPtr` (later made structural).
- **A Caught False Positive.** An exists-trace probe sharing only the payment hash appeared to show “value creation”. Tightening it to a single linked payment chain falsified it—the apparent attack was invoice-hash reuse across two unrelated offers, not a soundness bug.

- **Reusable Channels (Boundary).** Stage 1 (structural single-HTLC-per-output via the `Free(ptr)` slot) ships and re-verifies 43/43. Stage 2 (return `Free(ptr)` on resolution) is a clean tractability boundary: all-traces safety lemmas verify (~ 45 s each), but every exists-trace witness fails to converge. Stage 1 is therefore the shipped model.

9 CONCLUSION AND FUTURE WORK

We delivered a modular, machine-checked model that integrates the Lightning two-party channel lifecycle with multi-hop HTLC routing and fees (56 distinct properties over five theories). That integration surfaced a *linear single-spend discipline*: two independent soundness gaps arising from encoding one-shot on-chain outputs as persistent facts, repaired uniformly by linear tokens, and generalizable into an auditable modeling invariant. We further established the minimality of the T3 timing assumption empirically, proved end-to-end value conservation, exposed a grieving DoS, reproduced the known wormhole attack as a fidelity check, and mapped where the prover stops scaling.

The protocol-level safety results confirm expectations—consistent with the composable cryptographic treatment of Lightning [9] and the model-checking results of Grundmann and Hartenstein [7], which our symbolic model complements at the routing layer. The durable contribution here is methodological—a modeling discipline that catches a class of silent encoding errors, together with an account of what it takes to keep Tamarin tractable on a stateful, time-locked, arithmetic protocol, in a regime where every claim (positive or negative) was executed rather than asserted.

Future work includes testing whether the persistent-versus-linear pitfall is latent in other published symbolic models of payment channels and blockchain protocols, which would establish the discipline as a class-level finding rather than a single-model observation; extending the framework to Point Time-Locked Contracts (PTLCs); and bridging to computational verification. Our value conservation proofs verify strict amount invariants, but do not enforce game-theoretic economic rationality (e.g., defending against timelock bribery), which remains outside symbolic reasoning.

REFERENCES

- [1] Lightning Network community. 2024. BOLT: Basis of Lightning Technology (Lightning Network Specifications). <https://github.com/lightning/bolts>.
- [2] Colin Boyd, Kristian Gjøsteen, and Shuang Wu. 2020. A Blockchain Model in Tamarin and Formal Analysis of Hash Time Lock Contract. In *2nd Workshop on Formal Methods for Blockchains (FMBC)*, 5:1–5:13.
- [3] Sergiu Bursuc and Steve Kremer. 2019. Contingent Payments on a Public Ledger: Models and Reductions for Automated Verification. In *European Symposium on Research in Computer Security (ESORICS)*. Springer.
- [4] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. 2017. A Comprehensive Symbolic Analysis of TLS 1.3. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM.
- [5] Christian Decker and Roger Wattenhofer. 2015. A Fast and Scalable Payment Network with Bitcoin Duplex Micropayment Channels. In *Stabilization, Safety, and Security of Distributed Systems (SSS)*. Springer, 3–18.
- [6] Danny Dolev and Andrew C. Yao. 1983. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory* 29, 2 (1983), 198–208.
- [7] Matthias Grundmann and Hannes Hartenstein. 2025. Model Checking the Security of the Lightning Network. arXiv:2505.15568. (Supersedes arXiv:2307.02342.)
- [8] Lewis Gudgeon, Pedro Moreno-Sanchez, Stefanie Roos, Patrick McCorry, and Arthur Gervais. 2020. SoK: Layer-Two Blockchain Protocols. In *Financial Cryptography and Data Security (FC)*. Springer, 201–226.
- [9] Aggelos Kiayias and Orfeas Stefanos Thyfronitis Litos. 2020. A Composable Security Treatment of the Lightning Network. In *IEEE 33rd Computer Security Foundations Symposium (CSF)*. IEEE, 334–349.

- [10] Matteo Maffei. 2024. *Cryptocurrencies, Lecture 11: Scalability (Payment Channels and Payment-Channel Networks)*. Lecture Notes.
- [11] Giulio Malavolta, Pedro Moreno-Sanchez, Aniket Kate, Matteo Maffei, and Srivatsan Ravi. 2017. Concurrency and Privacy with Payment-Channel Networks. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 455–471.
- [12] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. 2019. Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability. In *Network and Distributed System Security Symposium (NDSS)*. The Internet Society.
- [13] Patrick McCorry, Surya Bakshi, Iddo Bentov, Sarah Meiklejohn, and Andrew Miller. 2019. Pisa: Arbitration Outsourcing for State Channels. In *ACM Conference on Advances in Financial Technologies (AFT)*. ACM, 16–30.
- [14] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. 2013. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In *Computer Aided Verification (CAV)*. Springer, 696–701.
- [15] Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry. 2019. Sprites and State Channels: Payment Networks that Go Faster Than Lightning. In *Financial Cryptography and Data Security (FC)*. Springer.
- [16] Joseph Poon and Thaddeus Dryja. 2016. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. <https://lightning.network/lightning-network-paper.pdf>. Whitepaper.
- [17] Zach Smith, Hugo Jonker, Sjouke Mauw, and Hyunwoo Lee. 2022. Modelling Agent-Skipping Attacks in Message Forwarding Protocols. arXiv:2201.08686.